Teoria de Linguagens e Compiladores

Preparação para a prova

Roteiro de estudo • métodos • simulado
Resoluções comentadas das duas listas

Como usar este material

Comece pelo roteiro de estudo. Tente cada exercício antes de consultar a resolução e refaça os pontos em que errou. As tabelas de autômatos especificam as setas dos diagramas; as tabelas LL/LR apresentam as decisões e os rastreamentos passo a passo.

Parte 1 · Guia de preparação e simulado

Parte 2 · Lista 1: 15 exercícios resolvidos

Parte 3 · Segunda lista enviada: 16 exercícios resolvidos

Anexo · 79 derivações do exercício sobre ambiguidade

Material elaborado a partir dos oito PDFs enviados. O arquivo lista2.pdf traz o título interno “Terceira Lista”; sua numeração original foi preservada. Os marcadores laterais do leitor de PDF permitem navegar pelas partes e exercícios.

Preparação para a prova - Teoria de Linguagens e Compiladores

Este guia usa as seis provas enviadas e as duas listas. O arquivo lista2.pdf está identificado no próprio documento como **Terceira Lista**; a resolução correspondente mantém a numeração dos seus 16 exercícios. Os enunciados foram tratados como material de estudo. Não use notas manuscritas como gabarito sem verificar.

1. O que estudar primeiro

Nas três primeiras avaliações, há cinco questões de 2 pontos, com estrutura muito parecida. Nas três segundas avaliações, a estrutura também se repete. Isso orienta a preparação, mas não garante a distribuição da próxima prova.

Bloco	Habilidade recorrente	Onde praticar
Autômatos	Construção de AFD para uma descrição de linguagem	Lista 1, 8-11
Autômatos	Produto para união/interseção e minimização	Lista 1, 9, 12 e 15
Autômatos	Expressão regular → AFN usando construções	Lista 1, 13
Autômatos	AFN → AFD com fecho ε e subconjuntos	Lista 1, 14
Gramáticas	Forma normal de Chomsky	Segunda lista enviada, 5
Gramáticas	GLC → autômato de pilha	Segunda lista, 3-4
Análise sintática	FIRST, FOLLOW, recursão à esquerda e tabela LL	Segunda lista, 10-14
Análise sintática	Itens LR, ACTION/GOTO e execução do analisador	Segunda lista, 15-16

Se o tempo for curto: priorize esses procedimentos. Conjuntos, funções, grafos, derivações e criação de gramáticas são a base e aparecem nas listas, mas não ocupam questões independentes nas páginas de enunciados dessas seis provas.

2. Roteiro de oito sessões

Use sessões de 90-120 minutos. Comece com 10 minutos de recuperação sem consulta, faça exercícios por 60-80 minutos e use os últimos 20 minutos para corrigir e refazer o trecho errado.

1. **Conceitos e AFDs.** Revise ε, ∅, conjuntos e linguagens. Faça lista 1, 1-8; depois construa sozinho os quatro AFDs do exercício 11.

2. **Produto e complemento.** Faça lista 1, 9-10 e 12. Explique em voz alta por que os estados finais usam “ou” na união e “e” na interseção.

3. **ER → AFN → AFD.** Faça lista 1, 13-14. Mostre os fechos ε, mesmo que enxergue um AFD pequeno diretamente.

4. **Minimização e treino do primeiro bloco.** Faça lista 1, 15 e uma primeira avaliação antiga em 90 minutos, sem consulta. Corrija pelo método, não pela semelhança do desenho.

5. **GLCs, pilha e Chomsky.** Faça segunda lista, 1-7. Treine a direção em que o lado direito de uma produção é colocado na pilha.

6. **FIRST/FOLLOW e LL.** Faça segunda lista, 10-14. Reconstrua os conjuntos do zero, justificando cada símbolo.

7. **LR.** Faça segunda lista, 15-16. Separe cálculo de estados, preenchimento da tabela e simulação da entrada.

8. **Simulado misto e revisão.** Faça o simulado ao final deste guia. Refaça uma questão de cada assunto em que errou e, depois, uma segunda avaliação antiga cronometrada.

Se você só tiver um dia, faça as sessões 2, 3, 6 e 7 em blocos de 60 minutos e use 30 minutos para cada um dos temas Chomsky e pilha. Esse é um plano de revisão, não substitui o treino completo.

3. Fundamentos que evitam erros

• **Símbolo:** um elemento do alfabeto, como a.

• **Cadeia:** uma sequência finita de símbolos, como aab.

• **ε ou λ:** cadeia de comprimento zero.

• **Linguagem:** conjunto de cadeias.

• **∅:** linguagem sem nenhuma cadeia; **{ε}:** linguagem com uma cadeia. Portanto, $\emptyset \neq \{\varepsilon\}$.

• **Σ*:** todas as cadeias finitas do alfabeto, incluindo ε.

• **R*:** zero ou mais repetições; **R^+:** uma ou mais; $R^+ = RR^*$.

• **Subcadeia:** trecho contíguo. ab é subcadeia de aabb; aba não é.

Um AFD é $(Q, \Sigma, \delta, q_0, F)$. A entrada só é aceita se terminar em F **depois de consumir a cadeia inteira**. Chegar antes a um estado final não encerra o processamento.

4. Como construir um AFD sem adivinhar

O estado representa a informação sobre o prefixo já lido que ainda pode influenciar a aceitação.

Contagem limitada

Para "pelo menos três 1s", use estados 0, 1, 2, 3, onde 3 significa **três ou mais**. Em 0 a contagem permanece; em 1 aumenta até saturar em 3. Inicial 0; final 3. Não é preciso guardar 4, 5, 6…

Paridade

Use P/I: ler o símbolo contado alterna a paridade; ler o outro mantém. A cadeia vazia tem zero ocorrências, portanto começa em P.

Subcadeia

Para detectar 00, os estados são "nenhum prefixo útil", "último símbolo é 0" e "já encontrei 00". O terceiro é absorvente. Para **não conter** 00, mantenha as mesmas transições e troque os finais, depois de garantir que o AFD está completo.

Para padrões maiores, guarde o maior sufixo do texto lido que também seja prefixo do padrão. No padrão 0110, após ler 0111, sobra apenas o prefixo vazio: nenhum sufixo não vazio é prefixo de 0110.

Testes de fronteira

Antes de entregar, teste: ε; uma cadeia mínima aceita; uma quase aceita; um símbolo repetido; uma cadeia que entra no estado final e depois sai dele.

5. Produto, complemento e minimização

Produto

Se M_1 e M_2 são completos sobre o mesmo alfabeto, o produto guarda (p,q) e faz:

$\delta((p,q), x) = (\delta_1(p,x), \delta_2(q,x))$.

• Inicial: $(\text{inicial}_1, \text{inicial}_2)$.

• União: final se $p \in F_1$ OU $q \in F_2$.

• Interseção: final se $p \in F_1$ E $q \in F_2$.

Exemplo: número par de 0s **ou** número par de 1s. Estados PP, PI, IP, II. Inicial PP. Os finais são PP, PI, IP. Em 0 troca a primeira letra; em 1 troca a segunda.

Complemento

Complete todas as transições, acrescentando um poço se necessário. Depois substitua F por Q−F. Inverter finais de AFN não calcula, em geral, complemento.

Minimização por partições

1. Remova estados inalcançáveis.

2. Separe finais e não finais.

3. Dentro de cada bloco, compare para quais blocos cada símbolo leva.

4. Separe estados com assinaturas diferentes e repita até estabilizar.

5. Cada bloco vira um estado do AFD mínimo.

Dois estados finais podem ser diferentes: talvez uma continuação leve um à rejeição e outro à aceitação. Para justificar uma separação, exiba essa continuação distinguível.

6. ER → AFN → AFD

Construções de AFNs

• Símbolo x: $p \xrightarrow{x} q$, com p inicial e q final.

• União: novo inicial com ε para os dois iniciais; finais dos dois ramos continuam finais.

• Concatenação: ε de cada final do primeiro para o inicial do segundo; só os finais do segundo continuam finais.

• Estrela: novo inicial final com ε para o antigo inicial; ε dos antigos finais para o antigo inicial; antigos finais continuam finais.

Use cópias com nomes distintos. Se o professor exigir a construção, não substitua a resposta apenas por um AFN intuitivo que aceita a mesma linguagem.

Subconjuntos com ε

$E(S)$ é o conjunto alcançável a partir de S usando zero ou mais transições ε. Inclui o próprio S.

1. Inicial do AFD: $E(\{\text{inicial do AFN}\})$.

2. Para cada conjunto T e símbolo x, calcule $E(\bigcup \delta(q,x), q \in T)$.

3. Crie um estado para cada conjunto novo e repita.

4. Um conjunto é final se contém **pelo menos um** final do AFN.

5. ∅, quando alcançado, é o poço do AFD.

Exemplo: AFN $0 \xrightarrow{\varepsilon} 1 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 3$, final 3. O inicial determinizado é {0,1}; ao ler a, chega a {2,3}, que é final. Não pare em {2}: falta fechar por ε.

7. Gramáticas e pilha

Uma GLC tem variáveis V, terminais Σ, produções P e símbolo inicial S. Na derivação mais à esquerda, substitua sempre a variável mais à esquerda; na mais à direita, a mais à direita.

Ambiguidade: uma cadeia com duas árvores de derivação diferentes. Duas ordens de expansão da mesma árvore não provam ambiguidade. Em E → E+E | E*E | a | b, b*a+a admite (b*a)+a e b*(a+a).

GLC → AP

Convenção deste material: topo da pilha à esquerda.

1. Inicialize a pilha com S Z.

2. Para cada A → α, permita substituir A por α sem consumir entrada.

3. Para cada terminal x, leia x e retire x do topo.

4. Aceite com entrada consumida e apenas Z restante, removendo Z e entrando no final.

Se estiver implementando empilhamentos unitários, empilhe α de trás para frente, para que o primeiro símbolo fique no topo. Recursão à esquerda não impede a construção de um AP não determinístico; apenas pode produzir ramos infinitos que não aceitam.

Forma normal de Chomsky

As regras finais têm formato A → BC ou A → a; admite-se $S_0 \to \varepsilon$ se ε pertence à linguagem, com S_0 fora de lados direitos.

Procedimento: novo inicial → anuláveis → eliminar ε preservando a cadeia vazia no novo inicial → eliminar regras unitárias → remover inúteis → isolar terminais em lados longos → binarizar.

Cuidado: ao remover uma variável anulável, considere todas as combinações de suas ocorrências. Para A → BAB com A e B anuláveis, aparecem BAB, AB, BA, BB, A, B, ε antes das limpezas.

8. FIRST, FOLLOW e LL(1)

FIRST

FIRST(α) contém os terminais que podem iniciar uma cadeia derivada de α, e ε se α pode desaparecer. Em ABC, só passe de A para B se A for anulável; só passe para C se A e B forem anuláveis.

FOLLOW

FOLLOW(A) contém terminais que podem vir imediatamente depois de A em uma forma sentencial, mais # se A puder terminar a entrada. Nunca contém ε.

- Coloque # em FOLLOW(inicial).
- Em X → αAβ, acrescente FIRST(β)−{ε} a FOLLOW(A).
- Se β for anulável ou vazio, acrescente FOLLOW(X) a FOLLOW(A).
- Repita até não haver alterações. Uma única passagem pode ser insuficiente.

Exemplo completo

S → aSBC | d, B → b | ε, C → c | ε.

FIRST(S)={a,d}; FIRST(B)={b,ε}; FIRST(C)={c,ε}.

FOLLOW(S)={#,b,c}, pois o S da direita é seguido de BC, que pode começar com b ou c e pode desaparecer. FOLLOW(B)={c,#,b}; FOLLOW(C)={#,b,c}. O b em FOLLOW(B) vem da propagação de FOLLOW(S), não de FIRST(C).

Recursão à esquerda

$A \to A\alpha_1 \mid A\alpha_2 \mid \beta$ vira:

$A \to \beta A'$

$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon$.

Isso não garante LL(1). Ainda podem existir conflitos de FIRST/FOLLOW.

Tabela LL

Para $A \to \alpha$, coloque a produção nas colunas de $FIRST(\alpha)-\{\varepsilon\}$. Se α for anulável, coloque-a também nas colunas de FOLLOW(A). Duas produções na mesma célula significam conflito: a gramática apresentada não é LL(1).

No analisador LL: topo terminal deve coincidir com o próximo símbolo; topo variável usa a tabela. Não consuma entrada ao apenas expandir uma produção.

9. LR: do ponto à execução

O item $A \to \alpha \cdot \beta$ indica que α já foi reconhecido e β ainda é esperado.

1. Aumente a gramática: $S' \to S$.

2. **closure:** se o ponto precede B, acrescente todos os itens $B \to \cdot\gamma$. Repita.

3. **goto(I,x):** avance o ponto sobre x em todos os itens que o permitem e calcule closure.

4. Repita até não surgirem conjuntos novos.

5. Transição em terminal é shift; em variável é GOTO; item completo é redução; $S' \to S\cdot$ aceita em #.

LR(0): reduções em todas as colunas de terminais e #. **SLR(1):** reduções só em FOLLOW do lado esquerdo. **LR(1):** usa lookaheads nos itens. Como os exercícios dizem genericamente LR(k), as resoluções identificam o método usado. As gramáticas dos exercícios 15 e 16 admitem LR(0).

Execução

Na pilha alternam-se estados e símbolos: 0 a 2 S 4. Shift empilha símbolo e estado e consome entrada. Reduzir $A \to \beta$ retira $2|\beta|$ entradas dessa representação, consulta GOTO no estado exposto e empilha A e o novo estado. A redução não consome a entrada.

Para $S \to aSc \mid b$, a linguagem é $\{a^n bc^n \mid n \geq 0\}$. Essa observação ajuda a prever o resultado, mas não substitui a tabela de execução solicitada.

10. Simulado misto - 100 minutos, 10 pontos

Faça sem olhar as resoluções. Cada questão vale 1 ponto.

1. Construa um AFD para cadeias binárias que não contêm 00.

2. Pelo produto, construa um AFD para número par de a e pelo menos um b.

3. Use as construções de união, concatenação e estrela para obter um AFN de $a(ab)^* \cup b$.

4. Determinize o AFN da questão 3, mostrando subconjuntos e fechos ε.

5. Minimize o produto para “pelo menos dois b OU exatamente dois a”.

6. Converta A → BAB | B | ε, B → 00 | ε para Chomsky.

7. Construa um AP para S → aSc | b.

8. Calcule FIRST/FOLLOW de A → ABCD | a, B → bB | ε, C → cC | ε, D → dD | ε.

9. Remova a recursão de S → Sc | SA | b, A → Aa | a e identifique se o resultado é LL(1).

10. Calcule os itens LR(0) de S → aSc | b e simule aabcc#.

Correção rápida do simulado

1. Três estados: seguro, último 0, proibido; os dois primeiros finais. Transições (0,1): seguro→(último0,seguro), último0→(proibido,seguro), proibido→(proibido,proibido).

2. Estados (paridade,viu_b), inicial (P,0), único final (P,1); a alterna paridade e b fixa o segundo componente em 1. 3-4. Veja a resolução da lista 1, exercícios 13.1-14.1, só depois da tentativa.

5. O mínimo tem 9 estados. Veja a partição justificada na lista 1, exercício 15.1.

6. A linguagem é um número par de zeros, incluindo ε; veja a conversão detalhada na segunda lista, exercício 5.

7. Inicialize SZ, expanda S por aSc ou b, case os terminais a,b,c e aceite ao remover Z com entrada consumida.

8. FIRST(A)={a}; FIRST(B)={b,ε}; FIRST(C)={c,ε}; FIRST(D)={d,ε}.
FOLLOW(A)=FOLLOW(B)=FOLLOW(C)=FOLLOW(D)={b,c,d,#}.

9. S → bR, R → cR | AR | ε, A → aT, T → aT | ε. Há conflito na célula (T,a).

10. Sequência de ações: s(a),s(a),s(b),r(S→b),s(c),r(S→aSc),s(c),r(S→aSc),aceita. Os estados estão na resolução da segunda lista, exercício 16.

11. Como saber se está pronto

Você deve conseguir, sem consulta: explicar o significado de cada estado que desenhou; calcular um fecho ε; justificar cada membro de FOLLOW; apontar o conflito de uma tabela LL; construir closure/goto LR; e executar uma redução sem consumir entrada.

Ao corrigir, registre o erro como uma regra concreta: “esqueci que ε pertence à estrela”, “não propaguei FOLLOW quando o sufixo era anulável”, “inverti o topo da pilha”. Refaça um exercício do mesmo tipo no dia seguinte.

Referências locais utilizadas

As páginas iniciais das seis provas fornecem os enunciados comparados; as duas listas foram lidas integralmente, incluindo figuras e tabelas.

- Provas/Prova 1/prova1_2022.pdf, prova1_2024.pdf, prova1_2025.pdf.
- Provas/Prova 2/prova2_2022.pdf, prova2_2023.pdf, prova2_2025.pdf.
- Listas de Exercicios/Lista 1/lista1.pdf e Listas de Exercicios/Lista 2/lista2.pdf.

Raiz desses caminhos: C:/Users/pedro/Desktop/Faculdade/2026-2/Teoria de Linguagens e Compiladores/.

Lista 1 - resolução comentada dos 15 exercícios

Fonte: lista1.pdf, Primeira Lista, Prof. Luiz Eduardo da Silva, UNIFAL-MG. As tabelas especificam todas as transições; podem ser usadas para desenhar os diagramas sem ambiguidade. Nos diagramas textuais, → marca o inicial e * marca um estado final. Transições não mencionadas de AFNs são vazias; em AFDs, todas são especificadas ou dadas por fórmula.

Convenções: ε=λ; $\mathbb{N}=\{0,1,2,\ldots\}$; alfabetos {a,b} nos exercícios 9-10 e 12-15. Se o curso usar $\mathbb{N}=\{1,2,\ldots\}$, ajuste os itens que dependem do zero.

Exercício 1 - descrições informais

1. $\{1,3,5,7,\ldots\}$: inteiros positivos ímpares.

2. $\{\ldots,-4,-2,0,2,4,\ldots\}$: inteiros pares.

3. $\{n \mid n=2m,\ m\in\mathbb{N}\}$: naturais pares, incluindo 0 na convenção adotada.

4. O enunciado imprime "2m em ℕ" e "2k em ℕ". Pela intenção usual, leia **m,k**∈ℕ: números naturais divisíveis por 2 e por 3, isto é, múltiplos de 6: $\{0,6,12,\ldots\}$. Literalmente, sem especificar o domínio de m e k, a escrita não garante essa interpretação; se m e k pudessem ser reais, as condições seriam diferentes. Não é correto corrigir esse detalhe silenciosamente.

5. $\{n \mid n\in\mathbb{Z} \text{ e } n=n+1\}$: conjunto vazio, pois subtrair n dos dois lados daria 0=1.

Exercício 2 - descrições formais

1. $\{3,47,80\}$.

2. $\{n\in\mathbb{Z} \mid n>7\}$.

3. $\{n\in\mathbb{N} \mid n<5\}=\{0,1,2,3,4\}$; sem 0, caso essa seja a convenção do professor.

4. $\{\varepsilon\}$. É um conjunto com um elemento.

5. $\emptyset$. É um conjunto com zero elementos.

Exercício 3 - operações de conjuntos

A={a,b,c}; B={a,b}.

1. A⊆B? **Não**, pois c∈A e c∉B.

2. B⊆A? **Sim**, pois a e b pertencem a A.

3. A∪B={a,b,c}.

4. A∩B={a,b}.

5. A×B={(a,a),(a,b),(b,a),(b,b),(c,a),(c,b)}. Os elementos são pares **ordenados**.

6. P(B)={∅,{a},{b},{a,b}}.

Exercício 4 - produto cartesiano

Para conjuntos finitos, **|A×B|=ab**. Cada um dos a primeiros componentes pode ser combinado com cada um dos b segundos componentes.

Exercício 5 - conjunto das partes

$|P(A)|=2^a$. Cada elemento de A oferece duas escolhas independentes: pertencer ou não ao subconjunto. Para A=∅, há um subconjunto: ∅; assim, $2^0=1$.

Exercício 6 - funções

O texto chama a função binária de f em um trecho, mas a tabela e os itens usam **g**. Adotamos g:X×Y→Y.

1. f(2)=7.

2. Domínio de f: X={1,2,3,4,5}; contradomínio: Y={6,7,8,9,10}. A imagem é {6,7}, que não deve ser confundida com o contradomínio.

3. g(2,10)=6, usando linha 2 e coluna 10.

4. Domínio de g: X×Y. Seus dois argumentos percorrem, respectivamente, X e Y. Contradomínio: Y.

5. f(4)=7; portanto g(4,f(4))=g(4,7)=**8**.

Exercício 7 - grafo não direcionado

Uma representação é:

```
   3
  / \
 1---2
  \ /
   4
```

As arestas são 1-2, 2-3, 1-3, 2-4 e 1-4. Não existe 3-4.

1. grau(1)=3: vizinhos 2,3,4.

2. grau(3)=2: vizinhos 1,2.

3. Um caminho de 3 a 4 é **3-1-4**; outro é 3-2-4.

Exercício 8 - diagrama a partir da definição formal

Inicial q_3, único final q_3. Desenhe cinco estados em linha. As setas u apontam para a esquerda e as d para a direita, com laços nas extremidades:

```
          d       d       d       d
q1 ---> q2 ---> →*q3 ---> q4 ---> q5
q1 <--- q2 <---  *q3 <--- q4 <--- q5
          u       u       u       u

laço u em q1; laço d em q5
```

Estado	u	d
q_1	q_1	q_2
q_2	q_1	q_3
$→^*q_3$	q_2	q_4

Estado	u	d
q_4	q_3	q_5
q_5	q_4	q_5

A cadeia vazia é aceita. As extremidades “saturam”; por isso não se deve descrever a linguagem apenas como cadeias com quantidades iguais de u e d.

Exercício 9 - interseções por produto

O “número para” do item 1 foi interpretado como **número par**. “Cada a é seguido por pelo menos um b” significa aqui que o símbolo imediatamente seguinte a cada a deve ser b. É a leitura usual dessa condição, equivalente a cadeias de $(b \cup ab)^*$.

Em todos os itens, construa os dois AFDs abaixo e faça o produto com:

$\delta((p,q),x)=(\delta_1(p,x),\delta_2(q,x))$.

Os finais são pares com **ambos** os componentes finais. Essa fórmula fornece todas as transições do AFD solicitado, inclusive dos pares que depois podem ser removidos por inacessibilidade.

9.1 Número par de a e cada a imediatamente seguido de b

Primeiro componente, paridade de a:

Estado	a	b
→*P	I	P
I	P	I

Segundo componente, obrigação de ler b:

Estado	a	b
→*L (livre)	W	L
W (aguarda b)	D	L
D (violação)	D	D

Produto: Q={P,I}×{L,W,D}, inicial (P,L), final **{(P,L)}**.

Estado	a	b
→*(P,L)	(I,W)	(P,L)
(I,L)	(P,W)	(I,L)
(P,W)	(I,D)	(P,L)
(I,W)	(P,D)	(I,L)
(P,D)	(I,D)	(P,D)
(I,D)	(P,D)	(I,D)

Aceita ε, b, abab; rejeita a, ab, aabb. Os dois estados com D podem ser fundidos se for pedida minimização.

9.2 Pelo menos três a e pelo menos dois b

AFD_1: estados A_0,A_1,A_2,A_3; inicial A_0; final A_3; a leva A_i a A_min(i+1,3), b mantém A_i.

AFD_2: estados B_0,B_1,B_2; inicial B_0; final B_2; b leva B_j a B_min(j+1,2), a mantém B_j.

Produto com **12 estados** (A_i,B_j), $i\in\{0,1,2,3\}$, $j\in\{0,1,2\}$:

• Inicial (A_0,B_0), único final (A_3,B_2).

• a: $(i,j) \to (\min(i+1,3),j)$.

• b: $(i,j) \to (i,\min(j+1,2))$.

Os índices guardam contagens saturadas. aaabb aceita; aabb e aaaab rejeitam.

9.3 Número par de a e um ou dois b

Use o AFD P/I do item 9.1. Para contar b, use B_0,B_1,B_2,B_3, em que B_3 significa três ou mais; inicial B_0, finais B_1 e B_2. Em a fica; em b incrementa saturando em 3.

Produto com 8 estados (p,B_j):

• Inicial (P,B_0); finais $\mathbf{(P,B_1),(P,B_2)}$.

• Em a troca P↔I e mantém j.

• Em b mantém p e troca j por min(j+1,3).

Aceita b, bb, aab, abab; rejeita ε, ab e bbb.

9.4 Pelo menos três a e termina em b

Primeiro componente $A_0…A_3$ do item 9.2. Segundo componente:

Estado	a	b
→N (vazia ou termina a)	N	B
*B (termina b)	N	B

Produto com 8 estados (A_i,t), inicial (A_0,N), único final $\mathbf{(A_3,B)}$. Em a incrementa i e vai a N; em b mantém i e vai a B.

Aceita aaab, baaab; rejeita aaa e aabb. O par (A_0,N) também representa a cadeia vazia.

Exercício 10 - complementos

Todas as tabelas a seguir são completas sobre {a,b}. Para cada item, as transições servem tanto para a linguagem simples quanto para seu complemento; só F muda.

10.1 Não contém baba

q_i, i<4, guarda o maior prefixo de baba que é sufixo do que já foi lido; q_4 significa padrão já encontrado.

Estado	a	b
→q_0	q_0	q_1
q_1 (b)	q_2	q_1
q_2 (ba)	q_0	q_3
q_3 (bab)	q_4	q_1

Estado	a	b
q_4 (encontrou)	q_4	q_4

Linguagem simples “contém baba”: F={q_4}. Solução “não contém baba”: **F={q_0,q_1,q_2,q_3}**. Em q_3 lendo b, sobra apenas o sufixo b: vai a q_1.

10.2 Não contém ab nem ba

Construa primeiro “contém ab ou ba”:

Estado	a	b
→s	A	B
A	A	D
B	D	B
D	D	D

F_simples={D}. A solução tem **F={s,A,B}**. Reconhece a* ∪ b*: só a, só b, ou ε. Repare que “nem” exige ausência das duas subcadeias.

10.3 Não está em a*b*

Linguagem simples a*b*:

Estado	a	b
→A	A	B
B	D	B
D	D	D

F_simples={A,B}. Complemento: **F={D}**. Equivale a “contém ba”, porque a ordem a…ab…b foi violada.

10.4 Não está em a* ∪ b*

Use a tabela do item 10.2. A linguagem simples tem finais {s,A,B}; a solução tem **F={D}**. Reconhece cadeias com pelo menos um a e pelo menos um b. ε não é aceita.

Exercício 11 - AFDs sobre {0,1}

As tabelas são também uma especificação de diagrama: uma seta por célula, inicial indicado por → e círculos duplos em *.

11.1 Começa com 1 e termina com 0

Estado	0	1
→s	D	U
U (começou 1, último 1)	Z	U
*Z (começou 1, último 0)	Z	U
D	D	D

Diagrama: →s -1→ U -0→ *Z -1→ U; laço 1 em U, laço 0 em Z; s -0→ D, com laços 0,1 em D. 10 aceita; ε, 0 e 1 rejeitam.

11.2 Pelo menos três 1s

Estado	0	1
→q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_3
*q_3	q_3	q_3

Diagrama: →q_0 -1→ q_1 -1→ q_2 -1→ *q_3, com laços 0 nos quatro estados e laço 1 em q_3.

11.3 Contém 0110

Estado	0	1
→q_0	q_1	q_0
q_1 (0)	q_1	q_2
q_2 (01)	q_1	q_3
q_3 (011)	q_4	q_0
*q_4 (encontrou)	q_4	q_4

Caminho principal: →q_0 -0→ q_1 -1→ q_2 -1→ q_3 -0→ *q_4. As demais setas estão na tabela. Observe q_2 -0→ q_1: o último 0 pode iniciar uma nova ocorrência.

11.4 Comprimento ≥3 e terceiro símbolo 0

Estado	0	1
→q_0	q_1	q_1
q_1	q_2	q_2
q_2	A	D
*A	A	A
D	D	D

Diagrama: →q_0 -0,1→ q_1 -0,1→ q_2; de q_2, 0 leva a *A e 1 a D; A e D têm laços 0,1. Depois do terceiro símbolo, o resto não altera a resposta.

Exercício 12 - operações sobre as figuras

Nomeie os estados da esquerda para a direita. A leitura das figuras dá:

Máquina	Estados	Inicial	Finais	Em a	Em b	Linguagem
M_1	B_0,B_1,B_2	B_0	B_2	mantém	incrementa até B_2	pelo menos dois b
M_2	A_0,A_1,A_2,A_3	A_0	A_2	incrementa até A_3	mantém	exatamente dois a
M_3	P,I	P	P	alterna	mantém	número par de a

Em M_2, A_3 é poço para a e b. Em M_1, B_2 também tem laços a,b, mas é final.

12.1 União por pares ordenados

$M_1 \cup M_2$. $Q=\{B_0,B_1,B_2\}\times\{A_0,A_1,A_2,A_3\}$; inicial (B_0,A_0). Finais: i=2 **ou** j=2. Todas as transições:

a: $(B_i,A_j) \to (B_i,A_min(j+1,3))$

b: $(B_i,A_j) \to (B_min(i+1,2),A_j)$.

Tabela completa dos 12 pares:

Estado (i,j)	a	b
→(0,0)	(0,1)	(1,0)
(0,1)	(0,2)	(1,1)
*(0,2)	(0,3)	(1,2)
(0,3)	(0,3)	(1,3)
(1,0)	(1,1)	(2,0)
(1,1)	(1,2)	(2,1)
*(1,2)	(1,3)	(2,2)
(1,3)	(1,3)	(2,3)
*(2,0)	(2,1)	(2,0)
*(2,1)	(2,2)	(2,1)
*(2,2)	(2,3)	(2,2)
*(2,3)	(2,3)	(2,3)

$M_1 \cup M_3$. $Q=\{B_0,B_1,B_2\}\times\{P,I\}$; inicial (B_0,P). Finais: i=2 **ou** paridade P.

Estado	a	b
→*(B_0,P)	(B_0,I)	(B_1,P)
(B_0,I)	(B_0,P)	(B_1,I)
*(B_1,P)	(B_1,I)	(B_2,P)
(B_1,I)	(B_1,P)	(B_2,I)
*(B_2,P)	(B_2,I)	(B_2,P)
*(B_2,I)	(B_2,P)	(B_2,I)

12.2 Concatenação por AFN

$M_1 \circ M_2$: use cópias disjuntas de B_0,B_1,B_2 e A_0,A_1,A_2,A_3. Mantenha as transições de ambas as máquinas e acrescente **$B_2 \xrightarrow{\varepsilon} A_0$**. Inicial B_0; único final A_2. B_2 deixa de ser final. Não elimine os laços de B_2.

$M_2 \circ M_3$: use $A_0 \ldots A_3$ e P,I, mantenha transições e acrescente **$A_2 \xrightarrow{\varepsilon} P$**. Inicial A_0; único final P. A_2 deixa de ser final.

Essas especificações enumeram todos os estados, iniciais, finais e transições. O não determinismo escolhe o ponto de separação da cadeia entre os fatores.

12.3 Estrela por AFN

M_1*: acrescente novo estado s, inicial e final, e setas s -ε→ B_0, B_2 -ε→ B_0. Mantenha todas as transições de M_1 e B_2 final. F={s,B_2}. O novo inicial é necessário para aceitar ε sem aceitar palavras que apenas chegaram a B_0 após uma repetição incompleta.

M_3*: acrescente novo estado t, inicial e final, com t -ε→ P e P -ε→ P; mantenha as transições e P final. F={t,P}. O laço ε em P é redundante, mas corresponde literalmente à construção aplicada a um final que já era inicial.

Conferência: L(M_1*)={ε}∪{w | w tem pelo menos dois b}; L(M_3*)=L(M_3), porque concatenar cadeias com paridade par preserva a paridade.

Exercício 13 - expressões regulares para AFNs

Usamos a construção com novo inicial para união; conexão ε para concatenação; e novo inicial final para estrela, preservando antigos finais e retornando por ε. Estados dos componentes são distintos. Em cada AFN abaixo, **as únicas transições são as listadas**.

13.1 a(ab)* ∪ b

Inicial s; finais {t,x3,r1}.

```
s  -ε→ p0, r0
p0 -a→ p1
p1 -ε→ t
t  -ε→ x0
x0 -a→ x1
x1 -ε→ x2
x2 -b→ x3
x3 -ε→ x0
r0 -b→ r1
```

O ramo p0-p1 lê o a inicial. t é o novo inicial final da estrela (ab)*. x0…x3 são a concatenação de a e b, e x3 é final com retorno para x0. O ramo r0-r1 aceita b. Aceita a, aab, aabab e b; rejeita ε e ab.

13.2 a^+ ∪ $(ab)^+$

Inicial s; finais {p1,q3}.

```
s  -ε→ p0, q0
p0 -a→ p1
p1 -ε→ p0
q0 -a→ q1
q1 -ε→ q2
q2 -b→ q3
q3 -ε→ q0
```

Para R^+, use o AFN de R com retorno ε dos finais ao inicial, preservando-os finais, **sem criar um inicial final para pular a primeira repetição**. É a construção de uma ou mais repetições, equivalente a RR*. Aceita a, aa, ab, abab; rejeita ε, b e aab.

13.3 a*(ab)b*

Inicial s; finais {z,v1}.

```
s  -ε→ p0, q0
```

```
p0 -a→ p1
p1 -ε→ p0, q0
q0 -a→ q1
q1 -ε→ q2
q2 -b→ q3
q3 -ε→ z
z  -ε→ v0
v0 -b→ v1
v1 -ε→ v0
```

A estrela inicial de a permite pular para q0; q0…q3 exige ab; z é o novo inicial final de b*. Ao concatenar, os finais dos componentes anteriores deixam de ser finais. A linguagem é a^+b^+. Aceita ab, aab e abbb; rejeita ε, a, b e aba.

Exercício 14 - determinização dos AFNs anteriores

Procedimento em todos os casos: inicial E({s}); em x, calcule E(move(T,x)); final se intersecta os finais do AFN. As tabelas abaixo incluem todos os subconjuntos alcançáveis, incluindo ∅.

14.1 Determinização de a(ab)* ∪ b

Estado/subconjunto	a	b
→D0 = {p0, r0, s}	D1	D2
*D1 = {p1, t, x0}	D3	D4
*D2 = {r1}	D4	D4
D3 = {x1, x2}	D4	D5
D4 = {}	D4	D4
*D5 = {x0, x3}	D3	D4

Exemplo de cálculo: E({s})={s,p0,r0}. Em a, move={p1}; seu fecho é {p1,t,x0}, final porque contém t. Daí, em a, o move é {x1} e o fecho é {x1,x2}; em b, esse último conjunto leva a {x0,x3}.

14.2 Determinização de $a^+ \cup (ab)^+$

Estado/subconjunto	a	b
→D0 = {p0, q0, s}	D1	D2
*D1 = {p0, p1, q1, q2}	D3	D4
D2 = {}	D2	D2
*D3 = {p0, p1}	D3	D2
*D4 = {q0, q3}	D5	D2
D5 = {q1, q2}	D2	D4

A palavra a deixa os dois ramos ainda possíveis; depois de aa, só o ramo a^+ continua. Não una estados apenas porque ambos são finais: suas continuações podem ser diferentes.

14.3 Determinização de a*(ab)b*

Estado/subconjunto	a	b
→D0 = {p0, q0, s}	D1	D2
D1 = {p0, p1, q0, q1, q2}	D1	D3
D2 = {}	D2	D2
*D3 = {q3, v0, z}	D2	D4
*D4 = {v0, v1}	D2	D4

Após ler pelo menos um a, tanto “continuar a estrela” quanto “usar esse a como início de ab” são possibilidades do AFN. O subconjunto guarda as duas; após o primeiro b, só b pode continuar.

Exercício 15 - minimização das uniões do exercício 12

15.1 $M_1 \cup M_2$

Todos os 12 pares são alcançáveis: para (i,j), use $a^j b^i$, interpretando i=2 e j=3 como limites saturados. Os quatro estados com i=2 são equivalentes, pois **qualquer continuação será aceita**. Una-os em U.

Nenhum outro par pode ser fundido. Denote C_{ij} os estados com i=0,1 e j=0,1,2,3.

Estado	a	b
→C_{00}	C_{01}	C_{10}
C_{01}	C_{02}	C_{11}
*C_{02}	C_{03}	C_{12}
C_{03}	C_{03}	C_{13}
C_{10}	C_{11}	U
C_{11}	C_{12}	U
*C_{12}	C_{13}	U
C_{13}	C_{13}	U
*U	U	U

Resultado: 9 estados.

Justificativa de minimalidade: U aceita todas as continuações; qualquer C_{ij} pode ser levado à rejeição acrescentando a suficientes e nenhum b. Para mesmo i e j distintos, complete uma das contagens de a para exatamente 2; se um j já vale 3, complete o outro. A continuação distingue os estados. Para i distintos, acrescente a suficientes para que os dois j ultrapassem 2 e, depois, um único b: o estado com i=1 atinge dois b e aceita, o com i=0 não. Logo os nove blocos são distinguíveis.

Uma sequência de refinamento também confirma o resultado: comece separando F/N; dentro dos finais, separe os com i=2 dos dois com j=2 e i<2; continue comparando destinos em a,b até restarem U e os oito C_{ij} acima.

15.2 $M_1 \cup M_3$

Una (B_2,P) e (B_2,I) em U, pois dois b já garantem aceitação independente da paridade.

Estado	a	b
→*C_0P	C_0I	C_1P
C_0I	C_0P	C_1I
*C_1P	C_1I	U
C_1I	C_1P	U
*U	U	U

Resultado: 5 estados. Finais {C_0P,C_1P,U}. A partição inicial é {C_0P,C_1P,U}/{C_0I,C_1I}. A leitura de a separa U dos outros finais; a leitura de b separa os dois não finais; com esses blocos distintos, os destinos em b separam C_0P e C_1P. A partição estabiliza em cinco blocos.

Conferência rápida: ε é aceita pela paridade par; a rejeita; ab rejeita; abb aceita por já conter dois b.

Segunda lista enviada - resolução comentada dos 16 exercícios

O arquivo se chama lista2.pdf, mas seu título interno é **Terceira Lista - Teoria de Linguagens e Compiladores**, Prof. Luiz Eduardo da Silva, UNIFAL-MG. Esta resolução corresponde aos 16 exercícios desse arquivo, sem renumerá-los.

Convenções: ε=λ; # é o marcador de fim da entrada; o **topo da pilha fica à esquerda** em APs e tabelas LL. Nas tabelas LR, a pilha cresce para a direita e alterna estados e símbolos. Uma célula vazia de tabela de análise significa erro.

Exercício 1 - gramática de expressões

```
E  → TE'
E' → +TE' | ε
T  → FT'
T' → *FT' | ε
F  → a | (E)
```

1. Variáveis: **{E,E',T,T',F}**.

2. Terminais: **{a,+,*,(,)}**. ε não é terminal.

3. Inicial: **E**.

1.4(a) Cadeia a

Mais à esquerda:

E ⇒ TE' ⇒ FT'E' ⇒ aT'E' ⇒ aE' ⇒ a.

Mais à direita:

E ⇒ TE' ⇒ T ⇒ FT' ⇒ F ⇒ a.

Árvore, em notação pai(filhos): E(T(F(a),T'(ε)),E'(ε)).

1.4(b) Cadeia a+a

Mais à esquerda:

E ⇒ TE' ⇒ FT'E' ⇒ aT'E' ⇒ aE' ⇒ a+TE' ⇒ a+FT'E' ⇒ a+aT'E' ⇒ a+aE' ⇒ a+a.

Mais à direita:

E ⇒ TE' ⇒ T+TE' ⇒ T+T ⇒ T+FT' ⇒ T+F ⇒ T+a ⇒ FT'+a ⇒ F+a ⇒ a+a.

Árvore: E(T(F(a),T'(ε)), E'(+,T(F(a),T'(ε)),E'(ε))).

1.4(c) Cadeia a+a+a

Mais à esquerda:

E ⇒ TE' ⇒ FT'E' ⇒ aT'E' ⇒ aE' ⇒ a+TE' ⇒ a+FT'E' ⇒ a+aT'E' ⇒ a+aE' ⇒ a+a+TE' ⇒ a+a+FT'E' ⇒ a+a+aT'E' ⇒ a+a+aE' ⇒ a+a+a.

Mais à direita:

E ⇒ TE' ⇒ T+TE' ⇒ T+T+TE' ⇒ T+T+T ⇒ T+T+FT' ⇒ T+T+F ⇒ T+T+a ⇒ T+FT'+a ⇒ T+F+a ⇒ T+a+a ⇒ FT'+a+a ⇒ F+a+a ⇒ a+a+a.

Árvore: E(T(F(a),T'(ε)), E'(+,T(F(a),T'(ε)),E'(+,T(F(a),T'(ε)),E'(ε)))).

1.4(d) Cadeia ((a))

Mais à esquerda:

```
E ⇒ TE' ⇒ FT'E' ⇒ (E)T'E'
  ⇒ (TE')T'E' ⇒ (FT'E')T'E' ⇒ ((E)T'E')T'E'
  ⇒ ((TE')T'E')T'E' ⇒ ((FT'E')T'E')T'E'
  ⇒ ((aT'E')T'E')T'E' ⇒ ((aE')T'E')T'E'
  ⇒ ((a)T'E')T'E' ⇒ ((a)E')T'E'
  ⇒ ((a))T'E' ⇒ ((a))E' ⇒ ((a))
```

Mais à direita:

E ⇒ TE' ⇒ T ⇒ FT' ⇒ F ⇒ (E) ⇒ (TE') ⇒ (T) ⇒ (FT') ⇒ (F) ⇒ ((E)) ⇒ ((TE')) ⇒ ((T)) ⇒ ((FT')) ⇒ ((F)) ⇒ ((a)).

Árvore completa em notação hierárquica:

```
E
├─ T
│  ├─ F
│  │  ├─ (
│  │  ├─ E
│  │  │  ├─ T
│  │  │  │  ├─ F
│  │  │  │  │  ├─ (
│  │  │  │  │  ├─ E
│  │  │  │  │  │  ├─ T
│  │  │  │  │  │  │  ├─ F → a
│  │  │  │  │  │  │  └─ T' → ε
│  │  │  │  │  │  └─ E' → ε
│  │  │  │  │  └─ )
│  │  │  │  └─ T' → ε
│  │  │  └─ E' → ε
│  │  └─ )
│  └─ T' → ε
└─ E' → ε
```

As representações pai(filhos) dos outros itens são árvores, não derivações: preservam todos os nós e a ordem dos filhos. Em cada caso, ler as folhas e omitir ε dá a cadeia pedida.

Exercício 2 - construir GLCs

2.1 Pelo menos três 1s

```
S → A1A1A1A
A → 0A | 1A | ε
```

Cada A gera qualquer cadeia binária; os três 1 explícitos garantem o mínimo. Toda cadeia com três ou mais 1 pode ser separada em quatro trechos por três desses 1.

2.2 Começa e termina com o mesmo símbolo

```
S → 0A0 | 1A1 | 0 | 1
A → 0A | 1A | ε
```

As regras S→0 e S→1 incluem as cadeias de comprimento 1. A cadeia vazia não tem primeiro/último símbolo e foi excluída; se uma convenção do exercício a incluir, acrescente S→ε.

2.3 Comprimento ímpar

```
S → 0 | 1 | 00S | 01S | 10S | 11S
```

Começa com um símbolo e acrescenta pares: comprimento 1+2k. Toda palavra ímpar pode ser decomposta assim.

2.4 Palíndromos

```
S → 0S0 | 1S1 | 0 | 1 | ε
```

Os símbolos externos são iguais; as bases cobrem comprimentos pares e ímpares. ε é palíndromo, pois $\varepsilon^R=\varepsilon$.

Exercício 3 - AP do diagrama

O desenho marca **q_0 e q_2 como finais**. Devemos preservar isso na transcrição.

Na notação do enunciado:

$A=(Q,\Sigma,\Gamma,\delta,q_0,F)$, com $Q=\{q_0,q_1,q_2\}$, $\Sigma=\{a,b\}$, $\Gamma=\{X,Z\}$, $F=\{q_0,q_2\}$; a pilha inicia em Z.

As transições do desenho são:

```
δ(q₀,a,ε) = {(q₀,X)}   # lê a e empilha X sem retirar o topo
δ(q₀,b,X) = {(q₁,ε)}   # primeiro b: retira X
δ(q₁,b,X) = {(q₁,ε)}   # outros b: retiram X
δ(q₁,ε,Z) = {(q₂,ε)}   # termina removendo o marcador
```

Na definição convencional que sempre consulta o topo, a primeira regra equivale a $\delta(q_0,a,\gamma)=\{(q_0,X\gamma)\}$ para $\gamma\in\{X,Z\}$; as demais permanecem iguais. Na 7-upla usual, acrescente explicitamente Z como símbolo inicial da pilha: $(Q,\Sigma,\Gamma,\delta,q_0,Z,F)$.

Processamento de aabb

Passo	Estado	Entrada restante	Pilha	Transição
0	q_0	aabb	Z	inicial
1	q_0	abb	XZ	lê a, empilha X
2	q_0	bb	XXZ	lê a, empilha X
3	q_1	b	XZ	lê b, retira X
4	q_1	ε	Z	lê b, retira X
5	q_2	ε	ε	ε, retira Z; aceita

Observação importante: por aceitação em estado final, exatamente como desenhado, a linguagem é $a^* \cup \{a^n b^n \mid n\geq 1\}$: qualquer cadeia só de a termina em q_0, que é final, mesmo com X na pilha. Se a intenção fosse apenas $\{a^n b^n \mid n\geq 0\}$, seria preciso alterar o desenho ou o critério de aceitação. Não se deve atribuir automaticamente essa linguagem ao AP original.

Exercício 4 - GLCs para APs

Para ambos, use três estados $\{i,q,f\}$, i inicial, f único final. A pilha começa vazia; alfabeto da pilha inclui variáveis, terminais e um marcador Z novo.

• Inicialização: $\delta(i,\varepsilon,\varepsilon)=\{(q,EZ)\}$.

• Para cada produção $A\to\alpha$, inclua (q,α) em $\delta(q,\varepsilon,A)$.

• Para cada terminal x, $\delta(q,x,x)=\{(q,\varepsilon)\}$.

• Finalização: $\delta(q,\varepsilon,Z)=\{(f,\varepsilon)\}$. Aceitação exige entrada toda consumida.

O primeiro símbolo de α fica no topo. Se as operações empilharem um símbolo por vez, faça-as da direita para a esquerda.

Primeira gramática

Σ={a,+,*,(,)}; Γ=Σ∪{E,E',T,T',F,Z}. Regras de expansão:

```
δ(q,ε,E)  = {(q,TE')}
δ(q,ε,E') = {(q,+TE'), (q,ε)}
δ(q,ε,T)  = {(q,FT')}
δ(q,ε,T') = {(q,*FT'), (q,ε)}
δ(q,ε,F)  = {(q,a), (q,(E))}
```

Acrescente as cinco regras de casamento de terminais, a inicialização e a finalização descritas acima. Isso especifica o AP inteiro.

Segunda gramática

Σ={a,+,×,(,)}; Γ=Σ∪{E,T,F,Z}. O símbolo de multiplicação nesta gramática foi escrito × no enunciado.

```
δ(q,ε,E) = {(q,E+T), (q,T)}
δ(q,ε,T) = {(q,T×F), (q,F)}
δ(q,ε,F) = {(q,(E)), (q,a)}
```

Acrescente casamento para cada terminal, inicialização e finalização. A recursão à esquerda pode criar ramos infinitos, mas o AP é não determinístico: basta existir um ramo finito aceitante.

Exercício 5 - forma normal de Chomsky

Original:

```
A → BAB | B | ε
B → 00 | ε
```

Etapa 1: novo inicial e anuláveis

Crie $S_0\to A$. São anuláveis A, B e S_0. A cadeia vazia deve continuar aceita, mas apenas S_0 terá produção ε na forma final.

De BAB, removendo ocorrências anuláveis independentemente, obtemos $BAB,AB,BA,BB,A,B,\varepsilon$ (algumas possibilidades duplicam a mesma palavra). Depois de remover ε e ignorar a regra inútil $A\to A$:

```
S₀ → A | ε
A  → BAB | AB | BA | BB | B
B  → 00
```

Etapa 2: remover unitárias

A→B copia 00 para A. S_0→A copia as produções não unitárias de A:

```
S₀ → BAB | AB | BA | BB | 00 | ε
A  → BAB | AB | BA | BB | 00
B  → 00
```

Etapa 3: terminais e binarização

Crie C→0 e X→AB. Substitua 00 por CC e BAB por BX:

```
S₀ → BX | AB | BA | BB | CC | ε
A  → BX | AB | BA | BB | CC
B  → CC
X  → AB
C  → 0
```

Essa é uma FNC válida: cada regra é duas variáveis, um terminal, ou a exceção S_0→ε; S_0 não ocorre à direita.

Conferência semântica: a linguagem original é $\{0^{2k} \mid k \geq 0\}$. Cada B acrescenta zero ou dois zeros; não aparecem comprimentos ímpares. Reciprocamente, A→BAB permite acrescentar dois zeros por repetição, mantendo o outro B vazio, e terminar em ε.

Uma FNC menor, também correta, é S_0→RR | CC | ε, R→RR | CC, C→0. Ela usa a caracterização da linguagem; a versão acima documenta a transformação pedida.

Exercício 6 - gramáticas para linguagens

6(a) $\{a^m b^n \mid m \geq n \geq 1\}$

```
S → aSb | A
A → aA | ab
```

Após k aplicações de S→aSb e r aplicações de A→aA, surge $a^k a^r ab\, b^k = a^{k+r+1} b^{k+1}$. Logo n=k+1≥1 e m=n+r≥n. Para quaisquer m,n válidos, escolha k=n−1 e r=m−n.

Exemplos: ab, aab, aabb, aaabb. Não gera b, abb ou ε.

6(b) $\{a^n b \mid n \geq 1\}$

```
S → aS | ab
```

A base fornece um a e um b; cada recursão acrescenta apenas a. Nunca surge mais de um b.

Exercício 7 - derivações de b*a+a

G_1: E→a | b | E+E | E*E | (E).

G_2: E→E+T | T, T→T*F | F, F→a | b | (E).

G_1: duas árvores diferentes

Árvore 1, **(b*a)+a**: E(E(E(b),*,E(a)),+,E(a)).

• Mais à esquerda: E ⇒ E+E ⇒ E*E+E ⇒ b*E+E ⇒ b*a+E ⇒ b*a+a.

• Mais à direita: E ⇒ E+E ⇒ E+a ⇒ E*E+a ⇒ E*a+a ⇒ b*a+a.

Árvore 2, **b*(a+a)**: E(E(b),*,E(E(a),+,E(a))).

• Mais à esquerda: E ⇒ E*E ⇒ b*E ⇒ b*E+E ⇒ b*a+E ⇒ b*a+a.

• Mais à direita: E ⇒ E*E ⇒ E*E+E ⇒ E*E+a ⇒ E*a+a ⇒ b*a+a.

Os parênteses em negrito indicam agrupamento da árvore; não são terminais dessas derivações. Duas árvores para a mesma cadeia provam que **G_1 é ambígua**.

G_2: uma árvore, precedência de *

Árvore: E(E(T(T(F(b)),*,F(a))),+,T(F(a))).

• Mais à esquerda: E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ b*F+T ⇒ b*a+T ⇒ b*a+F ⇒ b*a+a.

• Mais à direita: E ⇒ E+T ⇒ E+F ⇒ E+a ⇒ T+a ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a.

A soma só pode aparecer no nível E e a multiplicação no nível T. A cadeia tem uma única árvore em G_2 e significa (b*a)+a.

Sobre "enumere as derivações possíveis": considerando também ordens arbitrárias de substituição de variáveis, há 16 derivações em G_1 e 63 em G_2. Todas estão enumeradas no arquivo 04-anexo-derivacoes.md. As muitas ordens de G_2 não criam árvores novas. Se a intenção do professor for apenas derivações canônicas à esquerda/direita e ambiguidade, as seis sequências acima cobrem esse pedido.

Exercício 8 - duas sentenças

Gramática S→aS | bS | c.

• c, pela derivação S⇒c.

• abc, por S⇒aS⇒abS⇒abc.

A linguagem é {a,b}*c: exatamente um c, sempre no fim.

Exercício 9 - uma gramática qualquer

Escolha S→aS | bS | c, com terminais {a,b,c} e inicial S.

• ac: S⇒aS⇒ac.

• bac: S⇒bS⇒baS⇒bac.

Não é obrigatório usar cada terminal em toda sentença; o alfabeto da gramática é o conjunto dos terminais permitidos.

Exercício 10 - FIRST e FOLLOW com recursão e anuláveis

```
A → ABCD | a
B → bB | ε
C → cC | ε
D → dD | ε
```

A é o inicial. B,C,D são anuláveis; A **não** é, porque toda derivação terminal de A precisa usar A→a.

Variável	FIRST	FOLLOW
A	{a}	{b,c,d,#}
B	{b,ε}	{b,c,d,#}

Variável	FIRST	FOLLOW
C	{c,ε}	{b,c,d,#}
D	{d,ε}	{b,c,d,#}

Justificativa de FIRST

• a∈FIRST(A) por A→a. A→ABCD não acrescenta b,c,d porque o A inicial desse lado direito não é anulável.

• b∈FIRST(B) por B→bB; ε por B→ε.

• c∈FIRST(C) por C→cC; ε por C→ε.

• d∈FIRST(D) por D→dD; ε por D→ε.

Justificativa de FOLLOW

1. #∈FOLLOW(A), pois A é inicial.

2. Em A→A B C D, o A da direita é seguido de BCD. FIRST(BCD)−{ε}={b,c,d}; portanto os três entram em FOLLOW(A). BCD é anulável, propagando também FOLLOW(A) para ele mesmo, sem novos símbolos.

3. Para B nessa regra, o sufixo CD pode começar com c,d e pode desaparecer. Logo c,d entram diretamente, e todo FOLLOW(A)={b,c,d,#} é propagado para FOLLOW(B).

4. Para C, o sufixo D pode começar com d ou desaparecer: entra d e se propaga FOLLOW(A) inteiro.

5. D está no fim: recebe FOLLOW(A) inteiro.

6. Em B→bB, C→cC e D→dD, a variável final apenas recebe seu próprio FOLLOW; não surgem símbolos novos.

Assim, **b também pertence a FOLLOW(C) e FOLLOW(D)** por propagação. Ignorar essa propagação é o principal risco deste exercício.

Exercício 11 - recursão à esquerda e LL

Original: S→Sc | SA | b, A→Aa | a.

11(a) Remoção da recursão

Use R e T como novas variáveis:

```
S → bR
R → cR | AR | ε
A → aT
T → aT | ε
```

R repete os sufixos c ou A que antes eram produzidos pela recursão de S. T repete os a adicionais de A.

11(b) Conjuntos

Variável	FIRST	FOLLOW
S	{b}	{#}
R	{c,a,ε}	{#}
A	{a}	{a,c,#}
T	{a,ε}	{a,c,#}

FOLLOW(R) recebe # por S→bR. Em R→AR, A é seguido de R, que tem FIRST={a,c,ε}; por isso A recebe a,c e FOLLOW(R)={#}. T está no fim de A→aT e recebe FOLLOW(A).

Tabela preditiva

Variável	a	b	c	#
S	-	S→bR	-	-
R	R→AR	-	R→cR	R→ε
A	A→aT	-	-	-
T	**T→aT e T→ε**	-	T→ε	T→ε

Não é LL(1): conflito em **M[T,a]**. A regra T→aT entra pelo FIRST; T→ε entra porque a∈FOLLOW(T). Eliminar recursão não elimina necessariamente a ambiguidade entre consumir a dentro de A e começar outro A.

Como observação adicional, a linguagem é b{a,c}* e admite a gramática LL(1) equivalente S→bR, R→aR | cR | ε. Essa alternativa não muda a resposta sobre o conflito da gramática obtida pela transformação padrão.

Exercício 12 - FIRST e FOLLOW de S

Para S→aS | bS | c:

• **FIRST(S)={a,b,c}**, pelos três terminais iniciais das produções.

• **FOLLOW(S)={#}**, pois S é inicial e as demais ocorrências de S são finais de produções de S. Não há terminal depois delas.

ε não pertence a FIRST(S), porque toda sentença precisa terminar em c.

Exercício 13 - tabela LL e análise de abbac

Variável	a	b	c	#
S	S→aS	S→bS	S→c	-

Derivação mais à esquerda:

S ⇒ aS ⇒ abS ⇒ abbS ⇒ abbaS ⇒ abbac.

Análise, topo à esquerda:

Passo	Pilha	Entrada	Ação
0	S#	abbac#	S→aS
1	aS#	abbac#	casa a
2	S#	bbac#	S→bS
3	bS#	bbac#	casa b
4	S#	bac#	S→bS
5	bS#	bac#	casa b
6	S#	ac#	S→aS
7	aS#	ac#	casa a

Passo	Pilha	Entrada	Ação
8	S#	c#	S→c
9	c#	c#	casa c
10	#	#	aceita

Exercício 14 - gramática do else pendente

```
S  → iEtSS' | a
S' → eS | ε
E  → b
```

Variável	FIRST	FOLLOW
S	{i,a}	{e,#}
S'	{e,ε}	{e,#}
E	{b}	{t}

Justificativas: # entra em FOLLOW(S) por ser inicial; em S→iEtSS', o S interno recebe e por FIRST(S') e FOLLOW(S) porque S' é anulável. S' no fim recebe FOLLOW(S). E é seguido de t. A regra S'→eS também propaga FOLLOW(S') para FOLLOW(S).

Variável	i	a	b	t	e	#
S	S→iEtSS'	S→a	-	-	-	-
S'	-	-	-	-	**S'→eS e S'→ε**	S'→ε
E	-	-	E→b	-	-	-

Conflito em **M[S',e]**. Portanto a gramática apresentada não é LL(1). Escolher sempre o else para o if mais próximo é uma política de resolução; não torna essa tabela matematicamente livre de conflito.

Exercício 15 - coleção de estados e tabela LR

Numere as produções:

```
(0) D' → D        # aumentada
(1) D  → vL:t;
(2) L  → L,i
(3) L  → i
```

Variáveis originais: **{D,L}**; D' só pertence à gramática aumentada. Terminais: **{v,i,vírgula,dois-pontos,t,ponto-e-vírgula}**. Inicial: **D**. # é marcador externo, não terminal original.

Método

Construiremos **LR(0)**. Como não há conflitos, ele já atende a LR(k) com k=0. Se for utilizada a convenção SLR(1), os mesmos estados servem e as reduções ficam restritas a FOLLOW.

closure(I) acrescenta B→·γ sempre que há A→α·Bβ. goto(I,x) avança o ponto sobre x e fecha o conjunto.

Coleção canônica

```
I₀ = {D'→·D, D→·vL:t;}
I₁ = {D'→D·}
I₂ = {D→v·L:t;, L→·L,i, L→·i}
I₃ = {D→vL·:t;, L→L·,i}
I₄ = {L→i·}
I₅ = {L→L,·i}
I₆ = {D→vL:·t;}
I₇ = {L→L,i·}
I₈ = {D→vL:t·;}
I₉ = {D→vL:t;·}
```

Cálculos de goto não vazios:

```
goto(I₀,D)=I₁      goto(I₀,v)=I₂
goto(I₂,L)=I₃      goto(I₂,i)=I₄
goto(I₃,',')=I₅    goto(I₃,':')=I₆
goto(I₅,i)=I₇      goto(I₆,t)=I₈
goto(I₈,';')=I₉
```

Exemplo do fecho: avançar v em I_0 produz D→v·L:t;. Como o ponto precede L, acrescente L→·L,i e L→·i; não surge item novo após isso. Esse é I_2. Avançar L em I_2 move o ponto em **dois itens**, produzindo I_3.

ACTION/GOTO LR(0)

s_j=empilhar e ir ao estado j; r_i=reduzir pela produção i.

Estado	v	i	,	:	t	;	#	D	L
0	s2	-	-	-	-	-	-	1	-
1	-	-	-	-	-	-	aceita	-	-
2	-	s4	-	-	-	-	-	-	3
3	-	-	s5	s6	-	-	-	-	-
4	r3	r3	r3	r3	r3	r3	r3	-	-
5	-	s7	-	-	-	-	-	-	-
6	-	-	-	-	s8	-	-	-	-
7	r2	r2	r2	r2	r2	r2	r2	-	-
8	-	-	-	-	-	s9	-	-	-
9	r1	r1	r1	r1	r1	r1	r1	-	-

Não há estado com item completo e ação de shift, nem dois itens completos concorrentes: não há conflito.

Se a tabela pedida em aula usar **SLR(1)**, FOLLOW(D)={#} e FOLLOW(L)={vírgula,dois-pontos}; então mantenha r1 apenas na coluna #, e r2/r3 apenas nas colunas vírgula e dois-pontos. Todo o resto fica igual. Não misture esse preenchimento com a afirmação de estar fazendo LR(0).

Conferência: a linguagem é v i (,i)* : t ;, com uma ou mais ocorrências de i.

Exercício 16 - tabela fornecida, construção e rastreamentos

Produções: (1) S→aSc, (2) S→b. Gramática aumentada: S'→S. A linguagem é $\{a^n bc^n \mid n \geq 0\}$.

Construção dos estados

```
I₀ = closure({S'→·S}) = {S'→·S, S→·aSc, S→·b}
I₁ = goto(I₀,S)       = {S'→S·}
I₂ = goto(I₀,a)       = {S→a·Sc, S→·aSc, S→·b}
I₃ = goto(I₀,b)       = {S→b·}
I₄ = goto(I₂,S)       = {S→aS·c}
I₅ = goto(I₄,c)       = {S→aSc·}
```

Além disso, goto(I_2,a)=I_2 e goto(I_2,b)=I_3. Nos itens S→a·Sc de I_2, o ponto diante de S explica a inclusão das duas produções iniciadas por ponto. Esses são os estados e_0...e_5 da folha.

Tabela reconstruída - LR(0)

Estado	a	c	b	#	S
0	s2	-	s3	-	1
1	-	-	-	aceita	-
2	s2	-	s3	-	4
3	r2	r2	r2	r2	-
4	-	s5	-	-	-
5	r1	r1	r1	r1	-

“Símbolo reduzido” nas tabelas abaixo registra a produção usada na ação daquela linha; traço significa que não há redução. Pilha e entrada são mostradas **antes da ação**. Em r2 retire b e seu estado e empilhe S com GOTO; em r1 retire aSc e os três estados associados.

Entrada aaabc

Passo	Pilha antes da ação	Redução	Entrada restante	Ação
0	0	-	aaabc#	s2
1	0 a 2	-	aabc#	s2
2	0 a 2 a 2	-	abc#	s2
3	0 a 2 a 2 a 2	-	bc#	s3
4	0 a 2 a 2 a 2 b 3	S → b	c#	r2
5	0 a 2 a 2 a 2 S 4	-	c#	s5
6	0 a 2 a 2 a 2 S 4 c 5	S → aSc	#	r1
7	0 a 2 a 2 S 4	-	#	ERRO: célula vazia

Entrada aaabccc

Passo	Pilha antes da ação	Redução	Entrada restante	Ação
0	0	-	aaabccc#	s2
1	0 a 2	-	aabccc#	s2
2	0 a 2 a 2	-	abccc#	s2
3	0 a 2 a 2 a 2	-	bccc#	s3
4	0 a 2 a 2 a 2 b 3	S → b	ccc#	r2
5	0 a 2 a 2 a 2 S 4	-	ccc#	s5
6	0 a 2 a 2 a 2 S 4 c 5	S → aSc	cc#	r1
7	0 a 2 a 2 S 4	-	cc#	s5
8	0 a 2 a 2 S 4 c 5	S → aSc	c#	r1
9	0 a 2 S 4	-	c#	s5
10	0 a 2 S 4 c 5	S → aSc	#	r1
11	0 S 1	-	#	aceita

Entrada abbbc

Passo	Pilha antes da ação	Redução	Entrada restante	Ação
0	0	-	abbbc#	s2
1	0 a 2	-	bbbc#	s3
2	0 a 2 b 3	S → b	bbc#	r2
3	0 a 2 S 4	-	bbc#	ERRO: célula vazia

Entrada b

Passo	Pilha antes da ação	Redução	Entrada restante	Ação
0	0	-	b#	s3
1	0 b 3	S → b	#	r2
2	0 S 1	-	#	aceita

Entrada abc

Passo	Pilha antes da ação	Redução	Entrada restante	Ação
0	0	-	abc#	s2
1	0 a 2	-	bc#	s3
2	0 a 2 b 3	S → b	c#	r2
3	0 a 2 S 4	-	c#	s5
4	0 a 2 S 4 c 5	S → aSc	#	r1
5	0 S 1	-	#	aceita

Resultados e interpretação

Entrada	Resultado	Motivo
aaabc	rejeita	três a exigem três c, mas há só um
aaabccc	aceita	n=3
abbbc	rejeita	a linguagem tem exatamente um b
b	aceita	n=0
abc	aceita	n=1

Os erros dos rastreamentos são determinados pela tabela, não apenas pela contagem. Em aaabc o analisador fica esperando outro c; em abbbc encontra b quando o estado exige c.

Anexo - todas as derivações de b*a+a (exercício 7)

Enumeramos também ordens arbitrárias de expansão: algumas têm a mesma árvore. Só árvores distintas demonstram ambiguidade. As regras são as do PDF; não há produções ε.

G1 - 16 derivações

1. E ⇒ E+E ⇒ E*E+E ⇒ b*E+E ⇒ b*a+E ⇒ b*a+a
2. E ⇒ E+E ⇒ E*E+E ⇒ b*E+E ⇒ b*E+a ⇒ b*a+a
3. E ⇒ E+E ⇒ E*E+E ⇒ E*a+E ⇒ b*a+E ⇒ b*a+a
4. E ⇒ E+E ⇒ E*E+E ⇒ E*a+E ⇒ E*a+a ⇒ b*a+a
5. E ⇒ E+E ⇒ E*E+E ⇒ E*E+a ⇒ b*E+a ⇒ b*a+a
6. E ⇒ E+E ⇒ E*E+E ⇒ E*E+a ⇒ E*a+a ⇒ b*a+a
7. E ⇒ E+E ⇒ E+a ⇒ E*E+a ⇒ b*E+a ⇒ b*a+a
8. E ⇒ E+E ⇒ E+a ⇒ E*E+a ⇒ E*a+a ⇒ b*a+a
9. E ⇒ E*E ⇒ b*E ⇒ b*E+E ⇒ b*a+E ⇒ b*a+a
10. E ⇒ E*E ⇒ b*E ⇒ b*E+E ⇒ b*E+a ⇒ b*a+a
11. E ⇒ E*E ⇒ E*E+E ⇒ b*E+E ⇒ b*a+E ⇒ b*a+a
12. E ⇒ E*E ⇒ E*E+E ⇒ b*E+E ⇒ b*E+a ⇒ b*a+a
13. E ⇒ E*E ⇒ E*E+E ⇒ E*a+E ⇒ b*a+E ⇒ b*a+a
14. E ⇒ E*E ⇒ E*E+E ⇒ E*a+E ⇒ E*a+a ⇒ b*a+a
15. E ⇒ E*E ⇒ E*E+E ⇒ E*E+a ⇒ b*E+a ⇒ b*a+a
16. E ⇒ E*E ⇒ E*E+E ⇒ E*E+a ⇒ E*a+a ⇒ b*a+a

G2 - 63 derivações

1. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ b*F+T ⇒ b*a+T ⇒ b*a+F ⇒ b*a+a
2. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ b*F+T ⇒ b*F+F ⇒ b*a+F ⇒ b*a+a
3. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ b*F+T ⇒ b*F+F ⇒ b*F+a ⇒ b*a+a
4. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*a+T ⇒ b*a+T ⇒ b*a+F ⇒ b*a+a
5. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*a+T ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
6. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*a+T ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
7. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ b*F+F ⇒ b*a+F ⇒ b*a+a
8. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ b*F+F ⇒ b*F+a ⇒ b*a+a
9. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
10. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
11. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a

12. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ F*F+T ⇒ F*F+F ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
13. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ F*a+T ⇒ b*a+T ⇒ b*a+F ⇒ b*a+a
14. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ F*a+T ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
15. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ F*a+T ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
16. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ T*a+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
17. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ T*a+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
18. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*a+T ⇒ T*a+F ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
19. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*a+F ⇒ b*a+a
20. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*F+a ⇒ b*a+a
21. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
22. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
23. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
24. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
25. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
26. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
27. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*a+F ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
28. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
29. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
30. E ⇒ E+T ⇒ T+T ⇒ T*F+T ⇒ T*F+F ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
31. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*a+F ⇒ b*a+a
32. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*F+a ⇒ b*a+a
33. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
34. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
35. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
36. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
37. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
38. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
39. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
40. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
41. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
42. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
43. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
44. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
45. E ⇒ E+T ⇒ T+T ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
46. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*a+F ⇒ b*a+a
47. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ b*F+F ⇒ b*F+a ⇒ b*a+a

48. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
49. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
50. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
51. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ F*F+F ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
52. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ b*a+F ⇒ b*a+a
53. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ F*a+F ⇒ F*a+a ⇒ b*a+a
54. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*a+F ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
55. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
56. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
57. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T*F+F ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
58. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
59. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
60. E ⇒ E+T ⇒ E+F ⇒ T+F ⇒ T+a ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a
61. E ⇒ E+T ⇒ E+F ⇒ E+a ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ b*F+a ⇒ b*a+a
62. E ⇒ E+T ⇒ E+F ⇒ E+a ⇒ T+a ⇒ T*F+a ⇒ F*F+a ⇒ F*a+a ⇒ b*a+a
63. E ⇒ E+T ⇒ E+F ⇒ E+a ⇒ T+a ⇒ T*F+a ⇒ T*a+a ⇒ F*a+a ⇒ b*a+a